

Homework 4 Solutions

- 1. 线性回归 (4分)
- 2. 逻辑回归 (5分)
- 3. 逻辑回归: 分析 (6分)
- 4. 逻辑回归: 对抗攻击 (5分)
- 5. 向量化和伪代码 (6分)
- 6. 编程实验问题 (9分)
- 7. 编程 (65 分)
 - 7.1 任务要求
 - 7.2 特征模型
 - 7.3 feature.py
 - 7.4 lr.py
 - 7.5 命令行参数
 - 7.6 起始代码
- 8. 提交格式

1. 线性回归 (4分)

1. 希望通过最小化普通最小二乘 (OLS) 目标函数来用线性回归模型对数据集进行拟合

$$\mathcal{D} = \left\{ \left(\mathbf{x}^{(1)}, y^{(1)} \right), \left(\mathbf{x}^{(2)}, y^{(2)} \right), \dots, \left(\mathbf{x}^{(N)}, y^{(N)} \right) \right\}$$

其中, $\mathbf{x}^{(i)} \in \mathbb{R}^M$, OLS 目标函数为:

$$J(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - \sum_{j=1}^M w_j x_j^{(i)} \right)^2.$$

(a) (2 分) 单选: 通过从临界点 $\frac{\partial J(\mathbf{w})}{\partial w_k} = 0$ 推导出 w_k 的表达式来求解每个系数 $w_k (1 \leq k \leq M)$ 。请问在数据集 $(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})$ 上的 $w_1, \dots, w_{k-1}, w_{k+1}, \dots, w_M$ 中的每个 w_k 的表达式是什么?

- A. $w_k = \frac{\sum_{i=1}^N x_k^{(i)} (y^{(i)} - \sum_{j=1, j \neq k}^M w_j x_j^{(i)})}{\sum_{i=1}^N (x_k^{(i)})^2}$
- B. $w_k = \frac{\sum_{i=1}^N x_k^{(i)} (y^{(i)} - \sum_{j=1, j \neq k}^M w_j x_j^{(i)})}{\sum_{i=1}^N (y^{(i)})^2}$
- C. $w_k = \frac{\sum_{i=1}^N x_k^{(i)} (y^{(i)} - \sum_{j=1, j \neq k}^M w_j x_j^{(i)})}{\sum_{i=1}^N (x_k^{(i)} y^{(i)})^2}$
- D. $w_k = \sum_{i=1}^N x_k^{(i)} \left(y^{(i)} - \sum_{j=1}^M w_j x_j^{(i)} \right)$

答:

我们从给定的OLS目标函数开始:

$$J(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - \sum_{j=1}^M w_j x_j^{(i)} \right)^2$$

对 w_k 求偏导得:

$$\frac{\partial J(\mathbf{w})}{\partial w_k} = - \sum_{i=1}^N x_k^{(i)} \left(y^{(i)} - \sum_{j=1}^M w_j x_j^{(i)} \right)$$

将此式设为0来求解极小值, 得:

$$\sum_{i=1}^N x_k^{(i)} \left(y^{(i)} - \sum_{j=1, j \neq k}^M w_j x_j^{(i)} \right) = 0$$

为了解出 w_k , 我们需要找到一个形如 $w_k = \frac{\text{分子}}{\text{分母}}$ 的式子。我们可以这样操作: 将上面的式子左移项到右边, 然后两边同时除以 $\sum_{i=1}^N (x_k^{(i)})^2$ 。得到 w_k 的表达式为:

$$w_k = \frac{\sum_{i=1}^N x_k^{(i)} \left(y^{(i)} - \sum_{j=1, j \neq k}^M w_j x_j^{(i)} \right)}{\sum_{i=1}^N \left(x_k^{(i)} \right)^2}$$

故，选**A**。

(b) (1 分) 单选：需要估计多少个系数 w_k ？在求解这些系数时，可以列出多少个方程？

A. M 个系数, M 个方程

B. M 个系数, N 个方程

C. N 个系数, M 个方程

D. N 个系数, N 个方程

答：

本题中没有bias，所以总共有 M 个权重需要估计。

而又由于有 N 个样本点，所以一共能列出 N 个独立的方程。

故选**B**。

2. (1 分) 考虑一个数据集 D ，可以拟合一条线 $y = w_1 x + b_1$ 。设 $\bar{x}$ 和 $\bar{y}$ 分别为 x 和 y 坐标的均值。在对数据集进行均值中心化后创建 $D_{new} = ((x^{(1)} - \bar{x}, y^{(1)} - \bar{y}), \dots, (x^{(n)} - \bar{x}, y^{(n)} - \bar{y}))$ ，记 D_{new} 的线性回归解为 $y = w_2 x + b_2$ ，则 w_2 与 w_1 的大小关系是什么，并进行合理解释。

A. $w_2 > w_1$ 。均值中心化降低了偏置项的值，这使得回归线的斜率变大。

B. $w_2 < w_1$ 。均值中心化减少了数据的方差，所以回归线不会那么陡。

C. $w_2 = w_1$ 。均值中心化数据只是移动了数据，并没有缩放坐标；因此，它不会改变拟合回归线的斜率。

D. 没有足够的信息来进行判断。均值中心化可能会增加或减少标签值，所以它可能会使回归线更陡或者更平。

答：

权重 ω_1 和 ω_2 计算公式如下：

$$\omega_1 = \frac{\sum_{i=1}^n (x^{(i)} - \bar{x})(y^{(i)} - \bar{y})}{\sum_{i=1}^n (x^{(i)} - \bar{x})^2}$$

对于中心化后的数据 D_{new} ，我们有：

$$\omega_2 = \frac{\sum_{i=1}^n (x^{(i)} - \bar{x} - 0)(y^{(i)} - \bar{y} - 0)}{\sum_{i=1}^n (x^{(i)} - \bar{x} - 0)^2} = \omega_1$$

所以无论数据是否进行了均值中心化， ω_1 和 ω_2 实际上是相同的。均值中心化只是平移了数据，但并没有改变数据的分布或方差，因此回归线的斜率不会改变。

故选**C**。

2. 逻辑回归（5分）

下面的数据集包含 4 个训练样本，其中， $x_k^{(i)}$ 表示第 i 个训练样本 $\mathbf{x}^{(i)}$ 的第 k 维， $y^{(i)}$ 是相应的标签 ($k \in \{1, 2, 3\}$ 且 $i \in \{1, 2, 3, 4\}$)。

i	x_1	x_2	x_3	y
1	0	0	1	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0

在这个数据集上训练了一个二元逻辑回归模型，训练后的参数向量 θ 为

$$\theta = [1.5 \quad 2 \quad 1]^T$$

注意：在此问题中，没有使用截距项。

使用上述数据回答以下问题。对于所有数值答案，请四舍五入到小数点后四位；例如，0.1234。请写出你的解答过程。

1. (2 分) 计算 $J(\theta)$ ，即给定数据和参数 θ 上的负对数似然的 $1/N$ 。(注意我们在此使用的是自然对数，即底数为 e)。

答：

首先，可知推断过程的逻辑函数（sigmoid 函数）定义为：

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

而损失函数，在此为负对数似然函数的平均值：

$$J(\theta) = -\frac{1}{N} \sum_{i=1}^N [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

$J(\theta)$ 的计算结果为 **0.7975**。

2. (2 分) 计算所有 $j \in \{1, 2, 3\}$ 的梯度 $\frac{\partial J(\theta)}{\partial \theta_j}$ 关于 θ_j 。

答：

梯度计算公式为：

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

代入可知：

- $\frac{\partial J(\theta)}{\partial \theta_1} = \mathbf{0.2044}$
- $\frac{\partial J(\theta)}{\partial \theta_2} = \mathbf{-0.0417}$
- $\frac{\partial J(\theta)}{\partial \theta_3} = \mathbf{0.1709}$

3. (1 分) 按照 $\theta_j \leftarrow \theta_j - \eta \frac{\partial J(\theta)}{\partial \theta_j}$ 来更新参数，并写出向量 θ 的更新后 (数值) 值。这里学习率 $\eta = 1$ 。

答：

更新参数后，向量 θ 的新数值为：

$$\theta = [1.2956 \quad 2.0417 \quad 0.8291]^T$$

3. 逻辑回归：分析（6分）

1. (1 分) 多选: 关于逻辑回归, 以下哪些是正确的?

- A. 二元逻辑回归公式可以适用于连续的特征和离散的二元特征。
- B. 假设没有特征工程, 二元逻辑回归将在特征空间中形成一个线性决策边界。
- C. Sigmoid 函数是凸函数。
- D. 逻辑回归的负对数似然函数不是凸函数, 所以梯度下降可能会陷入次优局部最小值。
- E. 以上都不正确。

答：

A对。逻辑回归不受特征值类型的限制，它可以处理连续的和二元的特征。

B对。逻辑回归在特征空间中确实创建了一个线性的决策边界。

C错。Sigmoid函数自身不是凸的，但它在逻辑回归中用于映射预测值到(0,1)区间。其形状是一个"S"形状。

D错。逻辑回归的负对数似然函数是凸的，这意味着梯度下降可以找到全局最小值。

故选A和B。

2. (1 分) 单选: 二元逻辑回归的平均负对数似然 $J(\theta)$ 可以表示为：

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \left[-y^{(i)} \left(\theta^T \mathbf{x}^{(i)} \right) + \log \left(1 + \exp \left(\theta^T \mathbf{x}^{(i)} \right) \right) \right]$$

其中, $\mathbf{x}^{(i)} \in \mathbb{R}^{M+1}$ 是第 i 个样本点特征值的列向量, $y^{(i)} \in \{0, 1\}$ 是第 i 个样本的类别标签, $\theta \in \mathbb{R}^{M+1}$ 是权重向量。当想要进行逻辑岭回归 (即使用 ℓ_2 正则化) 时, 修改目标函数为

$$f(\theta) = J(\theta) + \lambda \frac{1}{2} \sum_{j=0}^M \theta_j^2$$

其中, λ 是正则化权重, θ_j 是权重向量 θ 中的第 j 个元素。假设使用学习率 η 更新 θ_k , 那么以下哪个是更新公式的正确表达?

- A. $\theta_k \leftarrow \theta_k + \eta \frac{\partial f(\theta)}{\partial \theta_k}$, 其中 $\frac{\partial f(\theta)}{\partial \theta_k} = \frac{1}{N} \sum_{i=1}^N \left[x_k^{(i)} \left(y^{(i)} - \frac{\exp(\theta^T \mathbf{x}^{(i)})}{1 + \exp(\theta^T \mathbf{x}^{(i)})} \right) \right] + \lambda \theta_k$
- B. $\theta_k \leftarrow \theta_k + \eta \frac{\partial f(\theta)}{\partial \theta_k}$, 其中 $\frac{\partial f(\theta)}{\partial \theta_k} = \frac{1}{N} \sum_{i=1}^N \left[x_k^{(i)} \left(-y^{(i)} + \frac{\exp(\theta^T \mathbf{x}^{(i)})}{1 + \exp(\theta^T \mathbf{x}^{(i)})} \right) \right] - \lambda \theta_k$

$$C. \theta_k \leftarrow \theta_k - \eta \frac{\partial f(\theta)}{\partial \theta_k}, \text{ 其中 } \frac{\partial f(\theta)}{\partial \theta_k} = \frac{1}{N} \sum_{i=1}^N \left[x_k^{(i)} \left(-y^{(i)} + \frac{\exp(\theta^T \mathbf{x}^{(i)})}{1 + \exp(\theta^T \mathbf{x}^{(i)})} \right) \right] + \lambda \theta_k$$

$$D. \theta_k \leftarrow \theta_k - \eta \frac{\partial f(\theta)}{\partial \theta_k}, \text{ 其中 } \frac{\partial f(\theta)}{\partial \theta_k} = \frac{1}{N} \sum_{i=1}^N \left[x_k^{(i)} \left(-y^{(i)} - \frac{\exp(\theta^T \mathbf{x}^{(i)})}{1 + \exp(\theta^T \mathbf{x}^{(i)})} \right) \right] + \lambda \theta_k$$

答：

$$\frac{\partial f(\theta)}{\partial \theta_k} = \frac{\partial J(\theta)}{\partial \theta_k} + \lambda \theta_k$$

对 $J(\theta)$ 部分，偏导数是：

$$\frac{\partial J(\theta)}{\partial \theta_k} = \frac{1}{N} \sum_{i=1}^N \left[x_k^{(i)} \left(-y^{(i)} + \frac{\exp(\theta^T \mathbf{x}^{(i)})}{1 + \exp(\theta^T \mathbf{x}^{(i)})} \right) \right]$$

代入可知，选**C**。

3. (2分) 如果存在一个阈值 t , 使得所有小于 t 的值有一个类标签, 所有大于或等于 t 的值有另一个类标签, 那么我们认为数据在一个维度上是可分的。如果你在至少一个维度上可分的训练数据上训练一个无正则化的逻辑回归模型, 并进行无限迭代, 那么相应的权重可能会在数值大小上变的无限大。这种现象的解释是什么? 请通过对公式的分析, 对此问题进行说明。

提示：思考一下如果训练一个无正则化的逻辑回归模型, 会发生什么情况, 以及权重在计算这些概率时的作用。

答：

梯度计算公式为：

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

其中 sigmoid 函数为 $h_{\theta}(x) = \frac{1}{1 + \exp(-\theta^T x)}$

参数更新公式为：

$$\theta_j \leftarrow \theta_j - \eta \frac{\partial J(\theta)}{\partial \theta_j}$$

考虑这种情况：所有的 $x < 0$ 的点都是负样本点($y = 0$)，所有 $x > 0$ 点都是正样本点($y = 1$)。

由于在 x 维度上可分，那么此时无论输入正负样本，都会得到 $\frac{\partial J(\theta)}{\partial \theta_j} < 0$ 的结果。那么无限迭代下去，权重就一定会无限大了。

4. (2 分) 多选: 正则化 (如 ℓ_1 和 ℓ_2) 如何帮助解决上一个问题中的现象?

- A. ℓ_1 正则化通过限制非零权重的数量, 防止权重无限大。
 B. ℓ_1 正则化通过将一些权重减小到 0, 有效地移除一些特征, 防止权重无限大。
 C. ℓ_2 正则化通过将一些权重的值减小到接近 0 (减小特征的效果, 但不一定移除它), 防止权重无限大。
 D. 以上都不正确。

答：

首先介绍两种正则化：

ℓ_1 正则化：这种正则化方式在损失函数中添加了权重的绝对值之和作为正则项，即 $\lambda \sum_{j=0}^M |\theta_j|$ 。这种正则化会导致一些权重变为 0，从而实现了特征选择。

ℓ_2 正则化：这种正则化方式在损失函数中添加了权重的平方和作为正则项，即 $\lambda \sum_{j=0}^M \theta_j^2$ 。这种正则化不会使权重变成 0，但会将它们减小到接近 0 的值，从而减少模型的复杂性。

现在来分析选项：

A错。虽然 ℓ_1 正则化会导致一些权重变成 0（即进行特征选择），但不是通过“限制非零权重的数量”实现的。

B对。 ℓ_1 正则化是能够进行特征选择，使得有些权重为0的。

C对。 ℓ_2 正则化不能使权重为 0，而只能将权重减小到接近 0 的值，从而降低模型的复杂性。

故选 **B** 和 **C**。

4. 逻辑回归：对抗攻击（5分）

图像可以用每个像素点对应的向量来进行数值表示。图像分类任务使用像素点对应的 向量作为特征来预测图像标签。

一个海洋保护团队通过要求参与者提交独角鲸的灰度图像来收集数据。每个像素具有 在连续范围 $[0, 1]$ 的强度值，零是最暗的。然后, 该组织运行一个逻辑回归模型来预测照片是否实际上包含独角鲸（在数学上, “独角鲸”的预测将对应于模型预测 $\mathbf{y}^{(i)} = 1$ ）。在一个训练数据集上对模型进行训练后, 在测试集

上的测试误差较高。该组织想要改进模型,并向那些用不包含独角鲸的照片但使模型却预测出照片包含有“独角鲸”(即,假正例)的人提供经济补偿。公司公布了他们学习的逻辑回归模型的参数。通过分析逻辑回归模型的参数,可以找出模型在哪些方面表现不佳或容易出错。例如,如果模型经常产生假正例(错误地将不包含独角鲸的照片分类为包含独角鲸),那么可以通过查看参数来理解为什么会这样,并可能找到优化模型的方法。具体来说,将使用梯度下降来实现这一点。

1. 给定公司的模型参数 θ (即,逻辑回归系数),对于一张图像,你定义了逻辑回归预测这张图片包含独角鲸的概率函数为

$$p(y = 1 \mid \mathbf{x}, \theta) = \sigma(\theta^T \mathbf{x})$$

其中 $\sigma(\cdot)$ 表示sigmoid函数。你希望优化 $\mathbf{x}$, 使模型生成一个“独角鲸”的预测。根据上面的描述:

- (a) (1分)写出概率函数关于特征值 $\mathbf{x}$ 的梯度。

答:

首先, sigmoid函数是:

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

其导数是:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

对于概率函数:

$$P(y = 1 \mid \mathbf{x}, \theta) = \sigma(\theta^T \mathbf{x})$$

梯度为:

$$\nabla_{\mathbf{x}} P(y = 1 \mid \mathbf{x}, \theta) = \sigma'(\theta^T \mathbf{x}) \theta = \sigma(\theta^T \mathbf{x})(1 - \sigma(\theta^T \mathbf{x})) \theta$$

- (b) (1分)写出用梯度上升对 $\mathbf{x}$ 进行更新的规则。

答:

梯度上升更新规则为:

$$\mathbf{x} \leftarrow \mathbf{x} + \alpha \nabla_{\mathbf{x}} P(y = 1 \mid \mathbf{x}, \theta)$$

其中 α 是学习率。

2. (1分) 考虑到特征值需要在 $[0, 1]$ 的范围内, 要求设计一个新的方法或策略来使模型生成一个“独角鲸”的预测, 而这个新方法或策略不应使用梯度计算。这意味着使用一种不涉及直接对特征进行梯度更新的方法, 来实现目标(即获得生成的独角鲸的图片)。

答:

可以直接使用模型参数:

既然模型的参数 θ 是已知的, 可以尝试找到一个 $\mathbf{x}$ 使得 $\theta^T \mathbf{x}$ 足够大, 从而令 $\sigma(\theta^T \mathbf{x})$ 接近 1。具体来说, 可以将 $\mathbf{x}$ 设置为 θ 的一个缩放版本, 确保 $\mathbf{x}$ 中的所有值都在 $[0, 1]$ 范围内。

$$\mathbf{x} = \frac{\theta - \min(\theta)}{\max(\theta) - \min(\theta)}$$

只要没有使用梯度更新算法, 那么不管方法多离奇, 均可给分。

3. (2分) 多选: 现在考虑一下逻辑回归是否适合这个任务。假设同样的白色独角鲸在深色海洋背景中被用来作为训练集。训练照片是在距离相机 30-50 米的地方, 以独角鲸的侧视图为中心拍摄的。以下哪些测试图像的描述, 模型会预测为“独角鲸”?

- A. 一个未出现在训练集中的新照片, 一只相同的独角鲸在图像的右上角。
B. 与训练集中的某一张照片相同, 但是照片中的独角鲸部分被等大的白色硬纸板剪裁的独角鲸替换。
C. 与训练集中的某一张照片相同, 但是背景变为白色。
D. 以上都不会。

答:

A错。如果没有对图片的空间位置进行预处理的话, 很可能识别不出来。因为单纯的逻辑回归模型并没有空间不变性。

B对。这个情况下, 模型可能会预测为“独角鲸”, 因为硬纸板的颜色和独角鲸相似, 而模型只能根据像素值进行预测。

C对。如果背景颜色变了, 模型的预测的确可能会受到影响, 但只要独角鲸部分没有变, 它仍然很有可能被预测为“独角鲸”。

故选 **B** 和 **C**。

5. 向量化和伪代码（6分）

假设所有 ndarrays 的 dtypes 都是 np.float64 , 向量是 1D 的 ndarrays 。

1. (2 分) 多选: 给定矩阵 $\mathbf{X} \in \mathbb{R}^{N \times M}$ 和向量 $\mathbf{v} \in \mathbb{R}^M$, 可以使用 NumPy 创建一个新的向量 $\mathbf{u} \in \mathbb{R}^N$, 其第 i 个元素是 $\mathbf{v}$ 和 $\mathbf{X}$ 的第 i 行之间的点积, 如下所示:

```
# X and v are numpy ndarrays
# X.shape == (N, M), v.shape == (M,)
u = np.zeros(X.shape[0])
for i in range(X.shape[0]):
    for j in range(X.shape[1]):
        u[i] += X[i, j] * v[j]
```

哪些操作会产生同样的结果?

- A. $\mathbf{u} = \mathbf{X} @ \mathbf{v}$
- B. $\mathbf{u} = \mathbf{v} @ \mathbf{X}$
- C. $\mathbf{u} = \text{np.matmul}(\mathbf{X}, \mathbf{v})$
- D. $\mathbf{u} = \text{np.matmul}(\mathbf{v}, \mathbf{X})$
- E. $\mathbf{u} = \mathbf{X} * \mathbf{v}$
- F. $\mathbf{u} = \mathbf{v} * \mathbf{X}$
- G. $\mathbf{u} = \text{np.dot}(\mathbf{X}, \mathbf{v})$
- H. $\mathbf{u} = \text{np.dot}(\mathbf{v}, \mathbf{X})$
- I. 以上都不对。

答:

ACG是正确顺序的矩阵乘法。

BD将正确的矩阵乘法的顺序搞反了。

EF都是逐元素乘法, 而不是矩阵乘法。

故选 A, C, 和 G。

2. 给定矩阵 $\mathbf{X} \in \mathbb{R}^{N \times M}$ 和向量 $\mathbf{w} \in \mathbb{R}^N$, 令 $\Omega = \sum_{i=0}^{N-1} w_i (\mathbf{x}_i - \overline{\mathbf{x}}_i)(\mathbf{x}_i - \overline{\mathbf{x}}_i)^T$, 其中 $\mathbf{x}_i \in \mathbb{R}^M$ 是表示 $\mathbf{X}$ 的第 i 行的列向量, $\overline{\mathbf{x}}_i \in \mathbb{R}$ 是 $\mathbf{x}_i$ 的均值, $w_i \in \mathbb{R}$ 是 $\mathbf{w}$ 的第 i 个元素 ($i \in \{0, 1, \dots, N-1\}$)。对于以下问题, 分别使用 $\mathbf{x}$ 和 $\mathbf{w}$ 代表 $\mathbf{X}$ 和 $\mathbf{w}$ 。

。 $\mathbf{X}.\text{shape} == (\mathbf{N}, \mathbf{M})$, $\mathbf{w}.\text{shape} == (\mathbf{N},)$ 。根据上述题干描述, 回答下面的问题:

- (a) (2 分) 多选: 选择有效的 Python 代码行, 构造一个矩阵, 其第 i 行是 $(\mathbf{x}_i - \overline{\mathbf{x}}_i)^T$ 。

- A. $(\mathbf{X} - \text{np.mean}(\mathbf{X}, \text{axis}=0)).T$
- B. $\mathbf{X} - \text{np.mean}(\mathbf{X}, \text{axis}=1, \text{keepdims}=\text{True})$
- C. $\mathbf{X} - \text{np.mean}(\mathbf{X}, \text{axis}=0, \text{keepdims}=\text{True})$
- D. $\mathbf{X} - \text{np.expand_dims}(\text{np.mean}(\mathbf{X}, \text{axis}=1), 1)$
- E. 上述选项全错。

答:

本题要得到如下的矩阵:

$$M_{ij} = x_{ij} - \frac{1}{M} \sum_{k=1}^M x_{ik}$$

AC错。在 $\mathbf{X}$ 的基础上, 减去当前行的平均值。首先用 np.mean 这一函数时, 想求每一行的均值, 应该选择对固定行的每一列求平均值, 故应设置参数 $\text{axis}=1$ 。

B对。因为我们还想保留这只剩1元素的维度, 还应该设置参数 $\text{keepdims}=\text{True}$ 。这样得到的矩阵, 就是 $M \times 1$ 的矩阵了。

D对。 $\text{np.mean}(\mathbf{X}, \text{axis}=1)$ 我们只能够得到维度为 M 的向量。于是通过 np.expand_dims , 后面的参数 1 其实是 $\text{axis}=1$ 的缩写, 表示将原本的 $(M,)$ 的向量给变成 $(M, 1)$ 的矩阵。

故选B和D。

- (b) (2 分) 单选: 假设 (a) 的结果存储在 M 中。选择有效的 Python 代码行, 根据 M 来计算 Ω 。

- A. $\text{np.matmul}(\mathbf{w} * \mathbf{M.T}, \mathbf{M})$
- B. $\text{np.matmul}(\mathbf{w} * \mathbf{M}, \mathbf{M.T})$
- C. $\text{np.dot}(\mathbf{w} * \mathbf{M}, \mathbf{M.T})$
- D. $\mathbf{w} * \text{np.dot}(\mathbf{M.T}, \mathbf{M})$
- E. 上述选项全错

答:

求 Ω 的公式如下:

$$\Omega = \sum_{i=0}^{N-1} w_i (\mathbf{x}_i - \overline{\mathbf{x}})(\mathbf{x}_i - \overline{\mathbf{x}})^T$$

* 运算符是数组之间的逐元素乘法。在这种操作中，两个数组必须具有“可广播”（broadcastable）的形状。

什么是可广播的形状？

- 1.如果某一维度的大小相同，或者其中一个数组在该维度的大小为1，则这两个数组在该维度上是兼容的。
- 2.如果在所有共有的维度上都是兼容的，则两个数组可以进行广播。

例如：

1. (4,) 和 (4,) ，逐元素相乘为 (4,)
2. (3, 4) 和 (4,) ，逐元素相乘为 (3, 4)
3. (3, 1) 和 (4,) ，逐元素相乘为 (3, 4)
4. (3, 4, 2) 和 (4, 2) ，逐元素相乘为 (3, 4, 2)
5. (4,1,2,1,3) 和 (2,3) ，逐元素相乘为 (4,1,2,2,3) 。

上述只有A选项是 (N,) 和 (M, N) 逐元素相乘，能够不报错得到 (M, N) 的结果，随后又 (M, N) 与 (N, M) 进行矩阵相乘，得到 (M, M) 的最终结果。

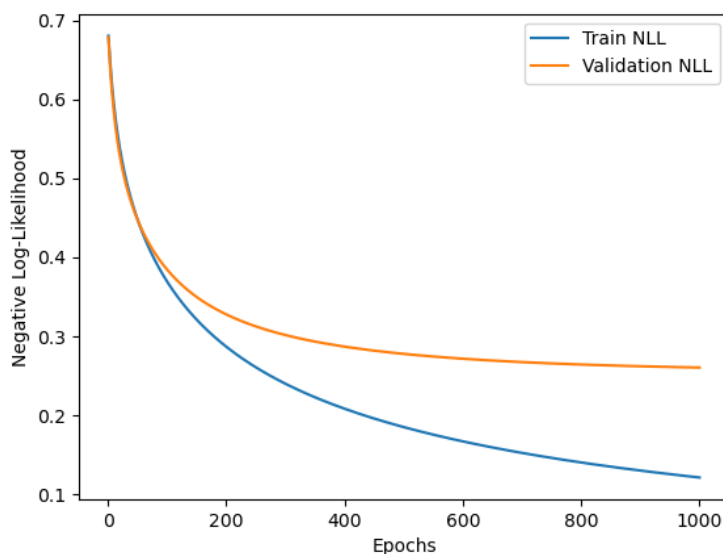
故选A。

6. 编程实验问题（9分）

以下问题应在完成此作业的编程部分时一并完成。请确保所有的图都是由计算机生成的。对于下面所有的问题，除非另有说明，都使用恒定的学习率 0.001。

- 1.（2分）使用 "handout/largedata" 文件夹里的数据，绘制一个图，展示经过 1,000 个周期（epochs）后，训练和验证数据集的平均负对数似然值。 y 轴应显示负对数似然值， x 轴应显示周期数（epochs）。

答：



- 2.（2分）写几句话解释上面实验的输出结果。特别是，训练和验证的对数似然曲线看起来相同还是不同？为什么？

答：

训练和验证的损失在初始阶段都在下降，但随着时间的推移，训练的损失会持续下降，而验证集的损失则是先下降然后一直稳定。

验证集没有上升，说明还没有过拟合。

- 3.（2分）在运行 1,000 个周期（epochs）后，报告你在 "handout/largedata" 文件夹中的大数据集上的训练和测试误差。请四舍五入到小数点后第四位，例如，0.1234。

答：

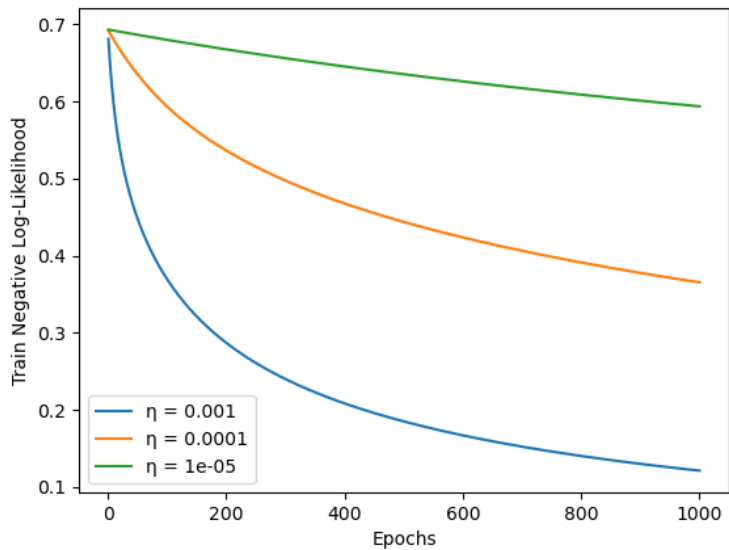
error(train): 0.0100

error(test): 0.1625

- 4.（2分）使用 "handout/largedata" 文件夹里的数据，绘制一个图来比较三个不同学习率 $\eta \in \{10^{-3}, 10^{-4}, 10^{-5}\}$ 下，经过各个周期（从 0 到 1,000 个周期）后的训练平均负对数似然值。 y 轴应显示平均负对数似然值， x 轴应显示周期数（从 0 到 1,000 个周期），图中应包含三条分别对应

三个不同 η 值的曲线。请提供一个图例（legend），指明每条曲线对应的学习率 η 。

答：



5. (1 分) 比较上一个问题中每条曲线的收敛速度有多快。

答：

在上一题中，越大的学习率，收敛速度也越快。

一般来说，较高的学习率会导致更快的收敛，但也有更高的风险陷入局部最优而不是全局最优。较低的学习率会导致更慢的收敛，但通常更容易找到全局最优解。

最大的学习率 10^{-3} 并没有导致不稳定的情况，没有在训练的过程中出现振荡。所以说选择这个学习率是比较合理的。

7. 编程（65 分）

本次作业的目标是使用二元逻辑回归实现一个有效的自然语言处理系统。你的算法要能判断一个餐厅的评论是积极的还是消极的。

注意：在开始编程之前，你应该先完成书面部分，以便对本编程部分有用的重要概念有一个良好的理解。

7.1 任务要求

数据集：已经包含在了 "handout" 文件夹内。这些数据来自于 Yelp¹ 数据集。在数据文件中，每一行是一个由标签（0 表示消极评论，1 表示积极评论）和一组单词组成的单一样本。每个样本（即每一行）的格式是 `label\tword1 word2 word3 ... wordN\n`，其中单词之间用空格分隔，标签和单词之间用制表符分隔。

¹Yelp数据集详细资料：<https://www.yelp.com/dataset>

数据示例如下：

```
1 i will never forget this single breakfast experience in mad...
0 the search for decent chinese takeout in madison continues...
0 sorry but me julio fell way below the standard even for med...
1 so this is the kind of food that will kill you so there s t...
```

特征工程：在课堂上，我们看到我们可以将逻辑回归应用于固定长度的实值输入（例如， $\mathbf{x}^{(i)} \in \mathbb{R}^n$ ）。然而，每条评论的长度不固定，也不是实值。

为了能在数据集上运行逻辑回归，我们首先需要使用一些基本的特征工程技术进行转换。在这次作业中，我们将使用一个词嵌入模型，在 7.2 部分中将详细描述。

程序：从总体来看，你将为这次作业编写两个程序：`feature.py` 和 `lr.py`。`feature.py` 接收原始输入数据，并为每个训练、验证和测试样本生成一个实值向量。`lr.py` 然后接收这些向量，并训练一个逻辑回归模型来预测每个样本是积极还是消极的评论。

7.2 特征模型

为了将一组单词转换成向量，我们依赖于一种流行的特征工程方法：词嵌入。

我们用 ϕ 表示一个特征工程方法，用 $\mathbf{x}^{(i)}$ 表示一个训练样本（一组英文单词，如在 7.1 节中所见）。

与仅仅指示哪些单词出现不同，词嵌入通过将每个单词“嵌入”到一个低维向量空间来表示它，这可能包含有关单词语义含义的更多信息。在这次作业中，我们使用 `word2vec` 嵌入，这是一组常用的特征向量。²

²这些嵌入向量更多的训练细节，参见原始论文：<https://arxiv.org/pdf/1301.3781.pdf>

嵌入：`word2vec.txt` 文件包含了 15k 个单词的 `word2vec` 嵌入。并非每条评论中的每个单词都出现在提供的 `word2vec.txt` 文件中。我们将这种缺失的单词视为“out-of-vocabulary”并忽略它们。每一行由一个单词和其嵌入组成，中间用制表符分

隔：`word\tfeature1\tfeature2\t...\tfeature300\n`。每一个单词的嵌入向量，一定是300维。举例如下，下面就是 `word2vec.txt` 文件中的几行：

```
films    -0.598  -0.622  -0.637  4.742   4.323   -5.980  ...
adapted  0.175   -0.399  -2.337  -0.299  -4.781  -2.029  ...
from     -0.114  -2.072  -0.874  -0.483   0.354  -2.205  ...
comic    -0.119  -1.952  -0.226  -0.825   5.625  -0.266  ...
```

词嵌入的使用：使用词嵌入对于这个模型，特征工程过程将有两个步骤：

1. 首先，我们希望从评论中排除不包含在 `word2vec` 字典中的单词。设 $\mathbf{x}_{\text{trim}}^{(i)} = \text{TRIM}(\mathbf{x}^{(i)})$ ，其中 $\text{TRIM}(\mathbf{x}^{(i)})$ 通过仅包括在 `word2vec.txt` 中出现的 $\mathbf{x}^{(i)}$ 的单词来修剪单词列表 $\mathbf{x}^{(i)}$ 。
2. 其次，我们希望将修剪后的向量 $\mathbf{x}_{\text{trim}}^{(i)}$ 转换为最终的特征向量，通过平均其单词的 `word2vec` 嵌入来实现：

$$\phi(\mathbf{x}^{(i)}) = \frac{1}{J} \sum_{j=1}^J \text{word2vec}(\mathbf{x}_{\text{trim},j}^{(i)})$$

其中 J 表示 $\mathbf{x}_{\text{trim}}^{(i)}$ 中的单词数量， $\mathbf{x}_{\text{trim},j}^{(i)}$ 是 $\mathbf{x}_{\text{trim}}^{(i)}$ 中的第 j 个单词。

在给定的方程中， $\text{word2vec}(\mathbf{x}_{\text{trim},j}^{(i)}) \in \mathbb{R}^{300}$ 是单词 $\mathbf{x}_{\text{trim},j}^{(i)}$ 的 `word2vec` 特征向量。

下面的例子提供了一个参考：

- 设 $\mathbf{x}^{(i)}$ 表示句子 "a hot dog is not a sandwich because it is not square"。
- 给定一个玩具 `word2vec` 字典如下：

```
hot       0.1    0.2    0.3
not       -0.1   0.2   -0.3
sandwich  0.0   -0.2   0.4
square    0.2   -0.1   0.5
```

- 然后， $\mathbf{x}_{\text{trim}}^{(i)}$ 表示修剪后的评论 "hot not sandwich not square"。在这个修剪后的文本中，不包含在 `word2vec` 字典中的单词被排除了。还要注意，我们保留了单词的顺序，并且在修剪后的文本中没有删除重复的单词。³

³保留重复词相当于按照词频对单词进行加权。如果“好”出现的频率是“坏”的3倍，那么这个电影评论更有可能是正面的而非负面的。

- $\mathbf{x}^{(i)}$ 的特征可以计算为：

$$\begin{aligned} \phi_2(\mathbf{x}^{(i)}) &= \frac{1}{5}(\text{word2vec}(\text{hot}) + 2 \times \text{word2vec}(\text{not}) + \text{word2vec}(\text{sandwich}) + \text{word2vec}(\text{square})) \\ &= [0.02, 0.06, 0.12]^T \end{aligned}$$

7.3 feature.py

`feature.py` 实现了词嵌入（如7.2节所描述）以将原始训练样本（一个标签和一个英文单词列表）转换为格式化的训练样本（一个标签和一个特征向量）。

输入：

- 用于训练、验证和测试的输入数据。每个数据点包含一个标签和一个按7.1节所描述的格式的英文餐厅评论。
- 用于词嵌入特征提取方法的 `word2vec` 嵌入。

输出：

- 用于训练、验证和测试的格式化数据。您应该对训练、验证和测试集中的每一个进行特征提取。

输出格式：

每个输出文件（一个用于训练数据，一个用于验证，还有一个用于测试）应该包含每个样本的格式化表示，每个样本打印在新的一行上。使用 `\n` 来创建新行。每一行的格式应该严格匹配 `label\tvalue1\tvalue2\tvalue3\t...\tvalueM\n`。

每一行对应于一个特定的餐厅评论，其中第一个条目是标签，其余的是特征向量中的特征。这些行是字典中所有单词的 `word2vec` 向量的总和。所有条目都用制表符隔开。资料夹中包含小数据集上的示例格式化输出，以下部分是您的参考。请将您的输出保留到小数点后6位。

1.000000	-0.166646	0.641027	-0.064805	...
0.000000	-0.224874	0.461526	-0.215232	...
0.000000	-0.222178	0.437475	-0.083073	...
1.000000	-0.215923	0.612535	0.061671	...

注意：请不要打乱训练集验证集测试集的顺序！否则会被系统判低分！

7.4 lr.py

`lr.py` 实现了一个逻辑回归分类器，该分类器接收格式化的训练数据并输出一个标签（要么是0，要么是1），该标签对应于每个餐厅评论是消极的还是积极的。

输入：

- 用于训练、验证和测试的格式化数据。每个数据点包含一个标签和相应的特征向量。这些文件是由 `feature.py` 生成的。
- 要训练的轮数（epoch），将作为命令行参数传入。
- 学习率，也通过命令行传入。

要求：

- 在你的模型中包括一个截距项。你可以将截距项视为一个单独的变量，或者将其折叠到参数向量中（推荐）。无论哪种情况，请确保正确地更新截距参数。
- 将所有模型参数初始化为0。
- 使用随机梯度下降（SGD）来训练逻辑回归模型。
- 按照输入文件中给出的数据的顺序对训练数据进行SGD更新。虽然我们通常会在SGD中随机打乱训练样本，但由于需要自动评分这个作业，我们需要训练过程是确定性的。请不要打乱训练数据。

输出：

- 训练和测试数据的标签。
- 训练和测试错误的指标。

输出标签格式：你的 `lr` 程序应该生成两个 `output.txt` 文件，包含你的模型在训练数据和测试数据上的预测。每个文件应该包含每个样例的预测标签，每个标签打印在新的一行上。这些文件的名称将作为命令行参数传入。使用 `\n` 来创建新行。下面给出了标签的一个示例。

```
1
0
0
1
```

输出指标格式：你的程序应该生成一个 `.txt` 文件，在训练完成后报告最终的训练和测试错误。这个文件的名称将作为命令行参数传入。

你报告的所有数字应该在参考解决方案的0.00001范围内，你应该将错误值四舍五入到小数点后6位。以下示例是在学习率为0.001的情况下，小数据集经过500个训练轮次后的参考解决方案。

```
error (train): 0.000000
error(test): 0.625000
```

输出文件中的每一行都应该以换行符 `\n` 结束。冒号后有一个空白字符。

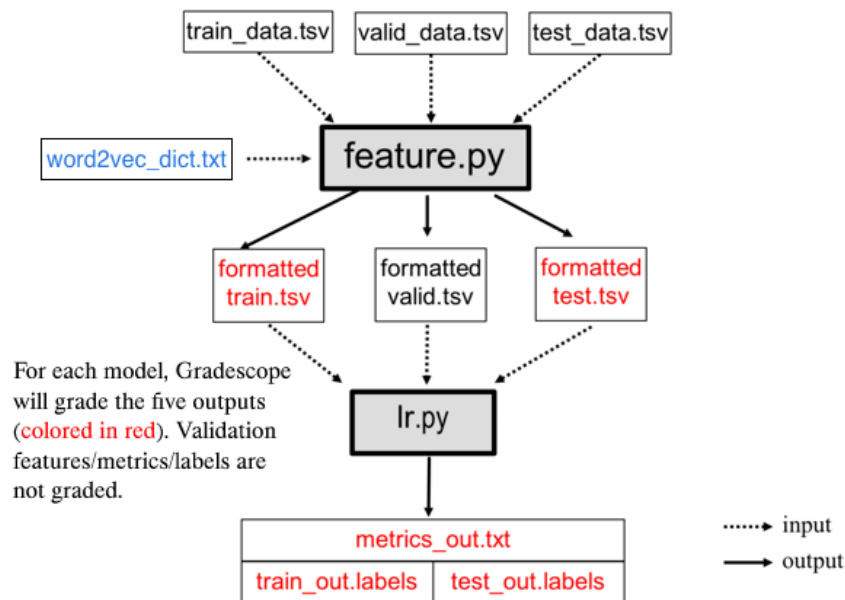


图 1：基于二元逻辑回归的情感分析器编程流程图

注意：请不要打乱训练集验证集测试集的顺序！否则会被系统判低分！

7.5 命令行参数

自动评分器使用以下命令运行并评估从生成的文件中得出的输出（注意，`feature.py` 将在 `lr.py` 之前运行）：

```
$ python feature.py [args1...]
$ python lr.py [args2...]
```

其中，`[args1...]` 是七个命令行参数的占位符：

1. `<train_input>`：指向训练输入 `.tsv` 文件的路径（参见第7.1节）
2. `<validation_input>`：指向验证输入 `.tsv` 文件的路径（参见第7.1节）
3. `<test_input>`：指向测试输入 `.tsv` 文件的路径（参见第7.1节）
4. `<feature_dictionary_input>`：指向 `word2vec` 特征字典 `.txt` 文件的路径（参见第7.2节）
5. `<formatted_train_out>`：指向应将训练数据上的特征提取写入的输出 `.tsv` 文件的路径（参见第7.3节）
6. `<formatted_validation_out>`：指向应将验证数据上的特征提取写入的输出 `.tsv` 文件的路径（参见第7.3节）
7. `<formatted_test_out>`：指向应将测试数据上的特征提取写入的输出 `.tsv` 文件的路径（参见第7.3节）

同样地，`[args2...]` 是八个命令行参数的占位符：

1. `<formatted_train_input>`：指向格式化训练输入 `.tsv` 文件的路径（参见第7.3节）
2. `<formatted_validation_input>`：指向格式化验证输入 `.tsv` 文件的路径（参见第7.3节）
3. `<formatted_test_input>`：指向格式化测试输入 `.tsv` 文件的路径（参见第7.3节）
4. `<train_out>`：指向应将训练数据上的预测写入的输出 `.txt` 文件的路径（参见第7.4节）
5. `<test_out>`：指向应将测试数据上的预测写入的输出 `.txt` 文件的路径（参见第7.4节）
6. `<metrics_out>`：输出 `.txt` 文件的路径，应将诸如训练和测试错误等指标写入（参见第7.4节）
7. `<num_epoch>`：指定SGD遍历所有训练数据的次数的整数（例如，如果`<num_epoch>`等于5，那么每个训练样本将在SGD中使用5次）。
8. `<learning_rate>`：指定学习率的浮点数；在参考输出中，我们将所有数据集的学习率设置为0.001。

例如，以下两条命令行将在手册中的大数据集上运行你的程序，进行500个周期。在"handout/largeoutput"和"handout/smalloutput"中，给出了这些命令的输出以及在小数据集上等效命令的输出。

```
$ python feature.py \
largedata/train_large.tsv \
largedata/val_large.tsv \
largedata/test_large.tsv \
word2vec.txt \
largeoutput/formatted_train_large.tsv \
largeoutput/formatted_val_large.tsv \
largeoutput/formatted_test_large.tsv
$ python lr.py \
largeoutput/formatted_train_large.tsv \
largeoutput/formatted_val_large.tsv \
largeoutput/formatted_test_large.tsv \
largeoutput/formatted_train_labels.txt \
largeoutput/formatted_test_labels.txt \
largeoutput/formatted_metrics.txt \
500 \
0.001
```

注意：不要在验证数据上写出预测，只在训练和测试数据上进行。验证数据仅用于在训练期间的每个周期结束时给出保留的负对数似然度估计。在“6 编程实验问题”部分，你要绘制验证和训练数据的负对数似然度与周期的图。（对于这个作业，我们总是会指定周期的数量。然而，更成熟的实现会在每个周期结束时监控验证数据的性能，并在此验证对数似然度似乎已经收敛时停止SGD。你不应该为这个任务实施这样的收敛检查。）

7.6 起始代码

为了帮助你开始这个作业，我们在 "handout" 中提供了起始代码。

8. 提交格式

1. 将问答部分的答案，整理成PDF格式，命名为 学号_姓名_HW4_Answer.pdf 。
2. 编程作业部分，只需提交 feature.py 和 lr.py，并将其分别重命名为 学号_姓名_feature.py 和 学号_姓名_lr.py 。（此处请格外注意！如果写错了，可能会不给分）
3. 将上述三个文件打包为不加密的 .zip 格式，并命名为 学号_姓名_HW4.zip 。